PORTFOLIO CASE STUDY · GRC ENGINEERING · HIPAA SECURITY RULE

A HIPAA compliance pipeline, built on a rebuilt Linux workstation

Seraphim Omisade · Capstone, CGE-P (GRC Engineering Practitioner)

SUMMARY

I built a deployment pipeline that will not ship a patient-intake application unless it meets HIPAA Security Rule safeguards, and that leaves signed, tamper-evident evidence an assessor can verify without trusting me.

5
HIPAA policies written as code

26
policy unit tests, all passing

11
live control tests, after every deploy and nightly

3 of 3
reviewer findings reproduced and fixed

CONSTRAINT
Six weeks total. Two of them went to rebuilding a workstation that could run the work, starting with a new battery for an older ThinkPad.

RESULT
Four enforced layers plus continuous monitoring. A test pull request that reintroduced a wildcard permission was blocked, and its evidence was still signed and preserved.

The pipeline



Apply runs only on **main** with zero policy failures. The job fails *last*, after evidence is stored, so a blocked change leaves the same signed record as an approved one.

Key details

MEASURE	VALUE
Evidence integrity	SHA-256 hashes, Cosign signature pinned to the workflow identity, S3 Object Lock retention
Primary framework	HIPAA Security Rule, because the app stores PHI
Platform rebuild	About 2 weeks: new battery, a $100 1TB drive, about 6 hours of OS trials

Why HIPAA comes first

The application stores patient intake records, so PHI is in scope and HIPAA is mandatory. SOC 2 and CMMC were deprioritized because they are driven by customer or contract opportunity, not by the data itself.

DECISIONS

Platform: Linux Mint on an upgraded drive

The pipeline runs on GitHub's Ubuntu runner, and Mint shares that base, so local results predict CI results.

OPTION	OUTCOME
macOS on an older Mac	REJECTED The OS version was too old for current tooling, and Homebrew installs stalled.
Windows with WSL	REJECTED Two file systems and two permission models doubled the troubleshooting surface.
Linux Mint on a new 1TB drive, in an older ThinkPad revived with a new battery	SELECTED Native bash and apt, same OS family as the CI runner.

The choice held up with one exception: default file permissions differed between Mint and the runner, which caused false drift alarms. The fix was one line:

TERRAFORM · HCL

```hcl
data "archive_file" "lambda" {
  type             = "zip"
  source_file      = "${path.module}/lambda/handler.py"
  output_path      = "${path.module}/lambda/handler.zip"
  output_file_mode = "0644"  # Mint creates 664, the CI checkout 644
}
```

WHAT I BUILT

Four enforced layers, then a fifth that keeps watching

1. **Infrastructure as code (Terraform).** Customer-managed KMS key with rotation, encrypted S3 and DynamoDB, CloudTrail with log file validation, and remote state.
2. **Policy as code (Rego with Conftest).** Five HIPAA policies evaluated against every Terraform plan, covered by 26 unit tests.
3. **CI/CD gate (GitHub Actions with OIDC).** Plan, Policy check, Apply, Sign, Upload. Apply runs only on main with zero failures, and the job fails last so blocked changes are still evidenced.
4. **Compliance documentation (OSCAL with compliance-trestle).** Validated component definition and profile, with each control linked to the evidence that proves it.
5. **Continuous monitoring.** AWS Config managed rules on a scoped recorder, EventBridge alerts, a nightly drift check under a read-only role, and 11 live control tests whose results go inside each signed evidence bundle.

BUILT WITH Linux Mint, Bash, Terraform, AWS (KMS, S3 Object Lock, DynamoDB, Lambda, CloudTrail, Config, EventBridge, IAM with GitHub OIDC), Open Policy Agent (Rego) and Conftest, GitHub Actions, Cosign and Sigstore, compliance-trestle (OSCAL), semgrep, and jq.

AN HONEST CORRECTION

During the OSCAL work I had marked HIPAA 164.312(d), person or entity authentication, as fully implemented. The intake API itself was unauthenticated, so I changed the claim to partial. The documentation has to match the system, not the goal.

POLICY AS CODE

The policy that blocked the test pull request

A simplified version of the least-privilege rule.

REGO · OPEN POLICY AGENT

```rego
package hipaa.iam_least_privilege
import rego.v1

# HIPAA 164.312(a)(1) Access control: no wildcard actions on inline role policies
deny contains msg if {
    some rc in input.resource_changes
    rc.type == "aws_iam_role_policy"
    doc := json.unmarshal(rc.change.after.policy)
    some stmt in as_array(doc.Statement)
    stmt.Effect == "Allow"
    some action in as_array(stmt.Action)
    is_wildcard(action)
    msg := sprintf("%s allows wildcard action %q", [rc.address, action])
}

is_wildcard(a) if a == "*"
is_wildcard(a) if endswith(a, ":*")

as_array(x) := x if is_array(x)
as_array(x) := [x] if not is_array(x)
```

ENGINEERING PROBLEMS SOLVED

Seven control failures in miniature

Seven issues surfaced between the first lab and the final run. I kept a record of each one, because each is a small version of a real control failure.

FINDING	FIX	WHY IT MATTERS
Home folder accidentally initialized as a git repo, with cloud credential folders inside	Removed the stray repo; added a global ignore for credentials and state	One push would have published cloud credentials
First pipeline run failed to assume its AWS role	Pinned OIDC trust to the repo's immutable subject (numeric owner and repo IDs)	A name can be re-registered by someone else; an ID cannot
Pipeline steps could report success on a failed apply	Ran every step under `bash -eo pipefail`	A green check on a failed deploy is false evidence
Semgrep HIGH: shell injection in the run-metadata step	Replaced `${{ }}` interpolation with runner environment variables and jq; pinned actions to commit SHAs	Untrusted input stops being executable code
False drift between laptop and CI	Pinned the Lambda archive's file mode to 0644	A drift alarm that cries wolf gets ignored
Evidence verification accepted any signer (`.*`)	Pinned verification to the pipeline workflow's identity	Proves who signed, not just that something was signed
Hand-built state bucket lacked a TLS-deny policy	Added the policy before enabling Config	Found my own gap before the scanner did

POST-CERTIFICATION REWORK

Using the second attempt for what it's for

The CGE-P allows a second attempt within 30 days. With the certificate already earned, I used that window to act on reviewer feedback. I reproduced each of the three findings before changing anything, then fixed all three. The reviewer was pointing at the distinction auditors draw between design and operating effectiveness: a compliant plan shows design, while live tests inside a signed bundle show the controls actually work.

FINDING	HOW I VERIFIED IT	WHAT I SHIPPED
Missing license; semgrep HIGH shell injection	Ran semgrep's shell-injection rule against the workflow; it flagged the exact line	MIT license crediting the original starter; injection fixed; the same rule then reported zero findings
No continuous monitoring	Confirmed the controls were checked only at plan time	AWS Config rules, EventBridge alerts, and a nightly drift run under a read-only role
No tests against the deployed system	Confirmed the evidence proved the plan, not the running controls	11 live tests: encryption, TLS denial, versioning, key rotation, CloudTrail validation, IAM simulation, and an end-to-end API call

KNOWN TRADE-OFFS

Documented in the repo, not hidden, each with a planned fix

State bucket outside the gate

Terraform can't keep its state in a bucket that doesn't exist yet, so the state bucket was bootstrapped by hand. It is versioned, private, TLS-only, and watched by Config, but outside the Rego suite. **Next: a small bootstrap configuration with its own policy run.**

Inline role policies and AWS Config

AWS Config's managed IAM policy rules evaluate customer-managed policies, not inline ones, so no managed rule checks inline role policies for wildcards. That control relies on the live IAM simulation and an alert. **Next: a custom Config rule.**

API authentication is partial

HIPAA 164.312(d) stays marked partial until the intake API requires authentication. **Next: authenticated intake API, then re-assess the claim.**

Apply role breadth

The deploy role could be scoped more tightly to the resource types the stack actually manages. **Next: narrow the role to managed resource types.**

WHAT THIS DEMONSTRATES

Regulation in, enforceable controls and verifiable evidence out

This project shows I can turn a regulation into controls that enforce themselves and evidence that holds up to an outside check.

Regulation to code

HIPAA safeguards expressed as Terraform defaults, Rego policies, and live tests, each mapped in OSCAL.

Evidence integrity

Hashed, signed, identity-pinned, retention-locked records, including for changes the gate rejected.

Honest control claims

Downgrading my own overclaim and documenting trade-offs instead of hiding them.

Troubleshooting

A rebuilt workstation and a pipeline that had to behave the same on a laptop and in CI, worked through from raw terminal output rather than guesswork. It's the same discipline I use in physical-layer field work.

Want the walkthrough?

A repository walkthrough and live evidence verification are available on request.

Email seraphim.omisade@proton.me
Portfolio seraphimo120.github.io
LinkedIn linkedin.com/in/seraphomi